

Justificativa da(s) escolha(s) de banco de dados

Projeto "Conexão Solidária" — plataforma da ONG Esperança Solidária · Hackathon

1. Critério de decisão

Este documento justifica a escolha de banco de dados a partir do **problema que o domínio de doações impõe** — volume de escrita esperado, requisitos de consistência e a garantia de que uma doação nunca seja contada duas vezes — e não a partir de conveniência de ferramenta. "O EF Core tem ótimo suporte a Postgres" é um fato real, mas é uma **decisão de tecnologia** (like escolher entre duas libs que fazem a mesma coisa), não uma **decisão de arquitetura** (qual categoria de banco o problema exige). As seções 2 a 4 tratam da decisão de arquitetura; a seção 5 trata da escolha específica de motor dentro da categoria relacional já decidida.

2. A carga real do domínio de doações

O enunciado do hackathon pede mensageria assíncrona porque, ao receber uma doação, a API **não** deve escrever o valor arrecadado direto no banco — deve publicar um evento e devolver a resposta na hora. Vale entender exatamente *que problema* isso resolve, porque é esse problema (e não a "escala" em abstrato) que orienta a escolha de banco:

- O gargalo que a fila resolve é **contenção no caminho síncrono do HTTP** — muitas doações chegando ao mesmo tempo não podem travar ou derrubar o request de quem está doando. Isso é resolvido devolvendo `202 Accepted` assim que o evento é publicado, antes de qualquer escrita de negócio acontecer.
- Quem efetivamente escreve o valor (`Donations.Worker`) faz isso **consumindo a fila de forma sequencial/controlada** — não é um enxame de requisições concorrentes brutas batendo no banco, é um processo que drena uma fila em ritmo previsível.
- Este não é um sistema de escala Big Tech: é uma plataforma de doações de uma ONG. Mesmo num pico "viral" de campanha, estamos falando de dezenas de doações por segundo, não milhares — e nem nesse cenário otimista o gargalo seria o banco, seria o processamento de pagamento externo (gateway de cartão/PIX), que nem faz parte deste MVP.

Conclusão da seção: o problema de escala deste domínio já foi resolvido pela fila, antes do banco entrar em cena. A pergunta que resta para o banco não é "aguenta o volume?" (um Postgres single-node aguenta milhares de transações por segundo, ordens de grandeza acima do que este domínio gera) — é "garante que cada doação seja aplicada exatamente uma vez, mesmo sob reentrega da fila?". É essa a pergunta que orienta a escolha.

3. O requisito central: consistência e idempotência sem duplicidade

Mensageria como RabbitMQ garante entrega **at-least-once** — nunca exactly-once. Isso significa que o mesmo `DonationReceivedEvent` pode, em cenários de falha/retry, chegar ao `Donations.Worker` mais de uma vez. Se o valor fosse simplesmente somado a cada mensagem recebida, uma reentrega dobraria o valor arrecadado de uma campanha — um bug financeiro grave, não cosmético. A arquitetura resolve isso empurrando a garantia de unicidade para o único lugar que pode oferecê-la de forma confiável sob concorrência: **uma constraint do banco dentro de uma transação ACID**, não uma checagem feita "à mão" no código da aplicação.

Como isso é aplicado, passo a passo, em `ApplyDonationUseCase` :

1. O evento carrega um `DonationId` único, gerado pela `Campaigns.Api` no momento em que a intenção de doação é aceita.

2. A tabela `donations.ProcessedDonations` usa exatamente esse `DonationId` como **chave primária** — não um id novo gerado pelo Worker.
3. Numa **única transação** (padrão Unit of Work / Template Method, replicado nos 3 serviços), o Worker faz duas coisas: incrementa `Campaign.AmountRaised` e insere a linha em `ProcessedDonations`.
4. Se a mensagem for reentregue, o segundo `INSERT` viola a chave primária já existente → a transação inteira sofre rollback → **o incremento de `AmountRaised` nunca é aplicado duas vezes**, porque ele estava na mesma transação que falhou.
5. Esse "conflito esperado" é classificado pelo `UnitOfWorkTemplate` (`TryClassifyAsConflict`) e devolvido como resultado, não como exception — o Worker simplesmente loga "já processado" e segue. Uma falha genuína (não relacionada a duplicidade) dispara retry exponencial e, se persistir, vai para a fila `_error` (DLQ) — nada é perdido silenciosamente.

O ponto central: a garantia de "nunca duplicar" não depende do código estar certo em todas as condições de corrida possíveis — depende de uma constraint que o banco fisicamente recusa violar. É o padrão "deixe o banco aplicar o invariante", só possível porque a transação combina leitura, escrita de negócio e checagem de unicidade como uma **operação atômica única**.

4. Por que um banco NoSQL seria overengineering aqui

Bancos NoSQL que escalam escrita horizontalmente (DynamoDB, Cassandra, MongoDB em cluster sharded) compram esse throughput abrindo mão exatamente da propriedade da seção 3: transação atômica multi-operação com isolamento forte entre múltiplos itens. Recriar a mesma garantia de idempotência nesses bancos exigiria:

- **Writes condicionais por item** (ex.: `ConditionExpression` do DynamoDB) para emular a unique constraint — já é mais código e mais casos de borda do que uma `PRIMARY KEY`.
- E, mesmo assim, **nenhuma atomicidade nativa** entre "incrementar o contador" e "registrar o ledger de idempotência" como uma única operação — a maioria desses bancos não oferece transação multi-documento com a mesma simplicidade/custo de um banco relacional (quando oferece, como o MongoDB moderno, é uma feature "pesada", adicionada por cima de um modelo pensado para não precisar dela).
- Isso empurraria a responsabilidade de consistência para **lógica de compensação na aplicação** (sagas, reconciliação, um serviço de idempotência à parte) — ou seja, reconstruir manualmente, com mais código e mais superfície de bug, o que o Postgres já garante de graça com `UNIQUE` + transação.

Motor NoSQL distribuído existe para sustentar escrita geograficamente distribuída em milhares de nós — um problema de escala real, mas que não é o desta plataforma (seção 2). Adotar um aqui seria pagar o custo de engenharia (mais complexidade operacional, uma tecnologia nova pra treinar/depurar/monitorar, garantias de consistência mais fracas a compensar manualmente) para resolver um problema de escala que não existe, enquanto se abre mão da propriedade que resolve o problema real do domínio (duplicidade). É o oposto do trade-off que faria sentido: trocar simplicidade e correção "de graça" por complexidade sem necessidade — a definição de overengineering.

5. Motor relacional escolhido: PostgreSQL

Com a categoria já decidida (relacional, pela seção 3), a escolha entre motores relacionais (Postgres, MySQL, SQL Server, etc.) é, sim, mais uma questão de tecnologia do que de arquitetura — mas vale registrar os critérios:

- **Código aberto e imagem Docker leve/madura** (`postgres:16`), sem custo de licença, adequado a um cluster Kubernetes local (kind) e ao prazo do hackathon.
- **Suporte nativo a múltiplos schemas dentro de um mesmo banco físico**, usado deliberadamente para separar a fronteira de propriedade de tabelas entre `Campaigns.Api` e `Donations.Worker` mesmo compartilhando a instância (ver seção 6) — schemas de primeira classe são um recurso mais maduro em Postgres do que em vários outros motores relacionais.
- **Ecossistema/tooling do time**: suporte de primeira classe no EF Core via `Npgsql.EntityFrameworkCore.PostgreSQL` (migrations, tipos ricos, transações ACID expostas diretamente). Isto facilita a implementação do padrão Unit of Work já decidido na arquitetura — mas reforça uma decisão de arquitetura já tomada, não é a razão dela.

6. Topologia: dois bancos físicos, não três

A decisão mais discutível da arquitetura é ter **dois** bancos físicos Postgres — `identity` e `campaigns` — em vez de um banco por serviço (três bancos, um também para `Donations.Worker`). Essa segunda opção seria o "database per service" mais estrito, mas foi descartada conscientemente.

Banco físico	Quem acessa	O que guarda
<code>identity</code> (db1)	Somente <code>Identity.Api</code>	Contas de usuário (<code>Donor</code> / <code>NgoManager</code>), hash Argon2id, claims de role
<code>campaigns</code> (db2)	<code>Campaigns.Api</code> (dono) + <code>Donations.Worker</code> (acesso restrito)	schema <code>public</code> : tabela <code>Campaigns</code> (dona: <code>Campaigns.Api</code>) schema <code>donations</code> : tabela <code>ProcessedDonations</code> (dona: <code>Donations.Worker</code>)

6.1 Por que o Worker não tem banco próprio

Como visto na seção 2, o requisito de mensageria assíncrona existe para desacoplar a escrita da doação do caminho síncrono da API — um problema de *contenção no request síncrono*, não um problema de isolamento físico de armazenamento. Como o `Donations.Worker` processa a fila de forma sequencial/controlada, completamente fora desse caminho síncrono, ele pode escrever com segurança direto no banco de `Campaigns.Api` sem reintroduzir a contenção que a fila já resolveu. Introduzir um terceiro banco físico só para o Worker adicionaria:

- mais um Postgres para provisionar, migrar e manter saudável no cluster (custo operacional sem benefício de consistência, já que o valor final mora logicamente em `Campaigns.Api` de qualquer forma);
- a necessidade de **sincronizar** o `AmountRaised` entre dois bancos diferentes — o que reintroduziria, entre bancos, o mesmo tipo de inconsistência distribuída que a seção 3 evita dentro de um único banco via transação atômica. Trocáramos um problema resolvido (contenção) por outro pior (consistência entre bancos, sem a rede de proteção de uma transação ACID única).

6.2 Como a fronteira de propriedade fica clara mesmo assim

Para não perder a clareza de "quem é dono de qual tabela" só porque a instância física é compartilhada, `Donations.Worker` usa um **schema Postgres próprio** (`donations`) dentro do banco `campaigns`:

- `donations.ProcessedDonations` — ledger de idempotência, schema e migrations **exclusivos** do Worker (`Donations.Infrastructure` é o único que gera migration para essa tabela).

- O Worker também enxerga a tabela `public.Campaigns` — mas só através de um repositório enxuto (`CampaignAmountRecord` , mapeando apenas `Status` e `AmountRaised`), marcado explicitamente com `ExcludeFromMigrations()` : o Worker nunca gera nem aplica migration nessa tabela, só lê e escreve nas colunas que precisa. A tabela continua migrada e "dona" por `Campaigns.Api` .
- `Campaigns.Api` , por sua vez, nunca escreve em `AmountRaised` — mesmo tendo acesso de leitura ao mesmo banco físico, essa regra é reforçada em código (nenhum use case ou repositório de `Campaigns.Api` toca essa coluna), não só por schema.

Resultado prático: mesma robustez de "quem escreve o quê" que um banco fisicamente separado daria, sem o custo de sincronizar dois bancos ou de reintroduzir contenção no caminho síncrono.

6.3 Por que `identity` continua isolado

Diferente do par `Campaigns.Api` / `Donations.Worker` , não há nenhuma razão de desempenho ou de fila para `Identity.Api` compartilhar banco com os outros dois — os dados de conta/credencial são de um domínio completamente diferente (autenticação), sensíveis (hash de senha), e não são lidos em runtime por nenhum outro serviço (o JWT já carrega as claims necessárias, assinado com RSA; `Campaigns.Api` só valida a assinatura com a chave pública, nunca consulta `Identity.Api` nem seu banco). Isolamento físico total aqui é estritamente melhor: menor superfície de acesso a dados sensíveis, e nenhum acoplamento a ganhar isolando.

7. Trade-offs considerados

Alternativa	Por que não foi escolhida
NoSQL (ex.: MongoDB, DynamoDB) para <code>Campaigns/Donations</code>	Resolve um problema de escala de escrita distribuída que este domínio não tem (seção 2), abrindo mão da transação atômica multi-operação que garante idempotência sem duplicidade de graça (seção 3) — exigiria reconstruir essa garantia manualmente (seção 4). Overengineering: mais complexidade, sem ganho.
3 bancos físicos (um por serviço, incluindo o Worker)	Reintroduz consistência distribuída entre bancos que a transação atômica única evita; custo operacional extra sem ganho de isolamento real, já que o dado final mora em <code>Campaigns.Api</code> de qualquer forma.
1 banco único para os três serviços	Perde o isolamento de <code>identity</code> , que é dado sensível de autenticação sem motivo de compartilhamento — nenhum ganho, só risco.

8. Conclusão

A escolha de banco desta plataforma é guiada pelo requisito de negócio mais crítico do domínio de doações — **nunca contar a mesma doação duas vezes, mesmo sob reentrega de mensagens** — e não por familiaridade com ferramenta. Um banco relacional com transações ACID entrega essa garantia através de uma simples `PRIMARY KEY` mais uma transação atômica; um NoSQL distribuído entregaria mais capacidade de escrita que este domínio não precisa (seção 2), ao custo de reconstruir manualmente a garantia de idempotência que já vem pronta (seção 4) — exatamente a definição de overengineering para este cenário. PostgreSQL foi então o motor relacional escolhido por critérios de tecnologia (seção 5), e a topologia de dois bancos físicos (`identity` isolado; `campaigns` compartilhado entre `Campaigns.Api` e `Donations.Worker` , com fronteira de propriedade reforçada por schema e por convenção de código) equilibra o requisito de mensageria assíncrona, a simplicidade operacional exigida pelo prazo do hackathon, e o isolamento de dados sensíveis onde ele realmente importa.

Conexão Solidária — documento gerado a partir de [docs/justificativa-banco.html](#) . Ver também [docs/arquitetura.png](#) e o [README.md](#) do repositório.